Select, Synchronisation, Context et Worker Pool

1. Le Multiplexeur `select` : L'aiguilleur de Channels

En programmation système C/Linux, les appels `select()`, `poll()` ou `epoll()` permettent de surveiller plusieurs descripteurs de fichiers (sockets réseau) en même temps. En Go, le mot-clé **`select`** applique exactement ce concept aux **Channels** : il permet à une goroutine d'attendre simultanément sur plusieurs opérations de lecture ou d'écriture de channels.

1.1 Règles de fonctionnement de `select`

Syntaxiquement proche d'un `switch`, chaque `case` d'un `select` doit obligatoirement être une opération sur un channel (`<-ch` ou `ch <- val`) :

1. **Attente bloquante :** Si aucun channel n'est prêt, `select` met la goroutine en pause jusqu'à ce que l'un des `case` devienne disponible.
2. **Équité pseudo-aléatoire :** Si **plusieurs** channels sont prêts exactement au même instant, le runtime Go tire au sort (pseudo-aléatoirement) lequel exécuter pour éviter toute famine (*starvation*).
3. **Mode Non-Bloquant (`default`) :** Si une clause `default:` est présente et qu'aucun channel n'est prêt, le bloc `default` s'exécute immédiatement sans jamais bloquer.

```
None
package main

import (
	"fmt"
	"time"
)

func main() {
	chRapide := make(chan string)
	chLent := make(chan string)

	go func() {
		time.Sleep(50 * time.Millisecond)
		chRapide <- "Réponse du cache Redis"
	}()

	go func() {
		time.Sleep(500 * time.Millisecond)
		chLent <- "Réponse de la base SQL"
```

```
	}()

	// 1. Multiplexage avec Timeout intégré (Pattern standard Go) :
	select {
	case msg := <-chRapide:
		fmt.Println("Gagnant :", msg)
	case msg := <-chLent:
		fmt.Println("Gagnant :", msg)
	case <-time.After(200 * time.Millisecond):
		fmt.Println("Erreur : Délai d'attente dépassé (Timeout) !")
	}
}
```

1.2 Astuce d'expert : Désactiver dynamiquement un `case` avec un channel `nil`

Rappelez-vous la matrice de vérité de J3S1 : **lire ou écrire sur un channel `nil` bloque à l'infini**. Dans un `select` placé dans une boucle `for`, affecter `ch = nil` à une variable channel permet de **désactiver dynamiquement ce `case`** pour le reste de la boucle sans avoir besoin de variables booléennes complexes !

2. Synchronisation et Partage Mémoire (`sync` & `-race`)

Si les Channels sont parfaits pour faire circuler des données d'une étape à une autre (*Pipeline*), utiliser un Channel juste pour incrémenter un compteur ou attendre la fin d'un groupe de goroutines est inutilement complexe et plus lent. Pour cela, Go fournit le package standard **`sync`**.

2.1 Attendre un groupe de Goroutines : `sync.WaitGroup`

Un `sync.WaitGroup` est un compteur atomique de synchronisation qui permet d'attendre qu'un ensemble de goroutines ait terminé son travail :

- **`wg.Add(n)` :** Incrémente le compteur de n goroutines à attendre. **Règle vitale : `wg.Add` doit TOUJOURS être appelé dans la goroutine parente AVANT l'instruction `go maFonction()` !** (Sinon, `wg.Wait()` risque de s'exécuter avant même que la goroutine n'ait eu le temps d'appeler `Add`).
- **`wg.Done()` :** Décrémente le compteur de `1` (toujours appelé avec **`defer wg.Done()`** au tout début de la goroutine lancée).
- **`wg.Wait()` :** Bloque jusqu'à ce que le compteur retombe exactement à `0`.

- **Règle mémoire absolue :** Un `sync.WaitGroup` (comme un `Mutex`) **ne doit JAMAIS être copié par valeur**. Si vous le passez en paramètre d'une fonction, passez **obligatoirement un pointeur `*sync.WaitGroup`** !

```
None
func executerLot() {
	var wg sync.WaitGroup

	for i := 1; i <= 5; i++ {
		wg.Add(1) // TOUJOURS avant le mot-clé 'go' !

		go func(id int) {
			defer wg.Done() // Garantit que le compteur diminuera même en
cas de return anticipé
			fmt.Printf("Worker %d a terminé\n", id)
		}(i)
	}

	wg.Wait() // Bloque ici jusqu'à ce que les 5 goroutines aient appelé Done()
	fmt.Println("Toutes les goroutines sont terminées !")
}
```

2.2 Les *Data Races* (Conditions de concurrence) et le détecteur `-race`

Puisque le runtime Go répartit vos goroutines sur plusieurs Threads OS (un par cœur CPU), **deux goroutines peuvent accéder à la même variable mémoire exactement à la même nanoseconde**.

Une **Data Race** se produit lorsque :

1. Au moins deux goroutines accèdent simultanément à la même adresse mémoire (variable, struct, slice, **ou `map` !**).
2. Au moins l'un de ces accès est une **écriture**.
3. Il n'y a aucune synchronisation entre elles.

Note critique sur les Maps : Contrairement aux idées reçues, **les `map` en Go ne sont PAS Thread-Safe !** Si deux goroutines écrivent simultanément dans la même `map` (ou si l'une lit pendant que l'autre écrit), le runtime Go déclenche un crash fatal **impossible à rattraper avec `recover()`** : `fatal error: concurrent map writes`.

L'arme absolue du compilateur Go : Le *Race Detector* (`-race`)

Go intègre directement le moteur *ThreadSanitizer* dans sa chaîne d'outils :

```
None
go run -race main.go
go test -race ./...
go build -race -o mon-serveur-preprod
```

Lorsque `-race` est activé, le compilateur instrumente chaque lecture et écriture mémoire. Si une *Data Race* se produit à l'exécution, Go affiche un rapport chirurgical indiquant quelles goroutines sont entrées en collision et à quelles lignes exactes !

2.3 Protéger l'état partagé : `sync.Mutex` et `sync.RWMutex`

Lorsqu'une structure de données en mémoire (un cache `map`, un compteur, un état de session) est partagée entre plusieurs goroutines, on l'encapsule avec un verrou d'exclusion mutuelle :

1. **`sync.Mutex` (Verrou exclusif) :** Une seule goroutine à la fois peut entrer dans la section critique entre `mu.Lock()` et `mu.Unlock()`.
2. **`sync.RWMutex` (Verrou Lecteurs / Rédacteur) :** Idéal quand il y a 95 % de lectures et 5 % d'écritures.
    - `mu.RLock()` / `mu.RUnlock()` : Autorise des **centaines de goroutines à lire simultanément** en parallèle.
    - `mu.Lock()` / `mu.Unlock()` : Bloque tous les lecteurs et donne un accès exclusif à un seul écrivain.

```
None
// Pattern idiomatique : embarquer le Mutex directement dans la struct à protéger
type CacheSecurise struct {
	mu      sync.RWMutex // Zero Value prête à l'emploi (pas besoin
d'initialisation !)
	donnees map[string]string
}

func (c *CacheSecurise) Ecrire(cle, valeur string) {
	c.mu.Lock()         // Verrouillage exclusif
	defer c.mu.Unlock() // Déverrouillage garanti à la sortie de la fonction
	c.donnees[cle] = valeur
}

func (c *CacheSecurise) Lire(cle string) (string, bool) {
	c.mu.RLock()         // Verrouillage partagé (plusieurs lecteurs en
parallèle)
	defer c.mu.RUnlock()
```

```
        val, ok := c.donnees[cle]
        return val, ok
}
```

Bonus : Pour un simple compteur entier (`int64`) ou un booléen d'état, le package `"sync/atomic"` (`atomic.Int64`, `atomic.Bool`) utilise directement les instructions matérielles du processeur sans verrouillage.

3. Contrôler le cycle de vie et éviter les fuites : `context.Context`

Règle fondamentale vue en J3S1 : **Une goroutine n'est jamais nettoyée par le Garbage Collector tant qu'elle n'a pas atteint son `return`.** Comment ordonner à 10 goroutines lancées pour traiter une requête HTTP de **s'arrêter immédiatement** si le client ferme son navigateur ou si un délai de 2 secondes est dépassé ?

En Go, on utilise le package standard **`context`**. Par convention absolue dans tout l'écosystème Go, **`ctx context.Context` est toujours le 1er paramètre** de toute fonction effectuant des entrées/sorties ou lançant des goroutines : `func MaFonction(ctx context.Context, arg string) error`.

3.1 Les 3 constructeurs de `context` d'annulation

On part toujours d'un contexte racine vide `context.Background()` (dans `main` ou à l'entrée du serveur), que l'on décore :

1. **`ctx, cancel := context.WithCancel(parentCtx)` :** Permet d'annuler manuellement toutes les goroutines enfants dès qu'on appelle la fonction `cancel()`.
2. **`ctx, cancel := context.WithTimeout(parentCtx, 500*time.Millisecond)` :** Annule automatiquement le contexte au bout de `500 ms` !
3. **`ctx, cancel := context.WithDeadline(parentCtx, heureLimite)` :** Annule automatiquement à une heure précise.

Règle d'or : Dès que vous créez un contexte avec `WithCancel` ou `WithTimeout`, vous devez **immédiatement** écrire **`defer cancel()`** à la ligne suivante pour libérer les timers internes du runtime !

3.2 Comment une Goroutine écoute-t-elle l'ordre d'arrêt ?

L'interface `context.Context` expose une méthode magique **`ctx.Done()`** qui retourne un channel en lecture seule (`<-chan struct{}`). Lorsque le timeout expire ou que `cancel()` est appelé, **le runtime Go ferme le channel `ctx.Done()`**, ce qui réveille instantanément toutes les goroutines qui l'écoutent dans un `select` !

```
None
func requeteLongue(ctx context.Context, id int) error {
	select {
	case <-time.After(2 * time.Second): // Simule un travail de 2 secondes
		fmt.Printf("[Goroutine %d] Travail terminé\n", id)
		return nil

	case <-ctx.Done(): // Déclenché instantanément si cancel() est appelé ou si
le Timeout expire !
		// ctx.Err() indique la raison exacte : context.Canceled ou
context.DeadlineExceeded
		fmt.Printf("[Goroutine %d] Arrêt d'urgence ordonné : %v\n", id,
ctx.Err())
		return ctx.Err() // La goroutine fait return -> Zéro Goroutine Leak !
	}
}
```

4. L'Architecture Reine en Production : Le *Worker Pool*

Lancer une goroutine par tâche sans limite (`for _, t := range millionDeTaches { go traiter(t) }`) est dangereux en production : si chaque goroutine ouvre une connexion vers votre base PostgreSQL ou appelle une API partenaire, vous allez provoquer un déni de service (DoS) sur votre propre infrastructure.

La solution universelle en Go est le **Worker Pool** (Groupe d'ouvriers borné), qui combine **toutes les notions du Jour 3** (`Goroutines` + `Channels` + `WaitGroup` + `Context`) :

4.1 Schéma et implémentation de référence d'un *Worker Pool*

```
None
package main

import (
	"context"
	"fmt"
	"sync"
	"time"
```

```
)

// Un Worker lit les tâches dans 'jobs', vérifie l'annulation 'ctx', et pousse
dans 'results'
func worker(ctx context.Context, id int, jobs <-chan int, results chan<- int, wg
*sync.WaitGroup) {
	defer wg.Done()

	for {
		select {
		case <-ctx.Done():
			// Signal d'arrêt global reçu : le worker s'arrête proprement
			return

		case tache, ouvert := <-jobs:
			if !ouvert {
				// Le channel 'jobs' a été fermé par main() et est vide
: travail terminé !
				return
			}
			// Traitement de la tâche
			time.Sleep(20 * time.Millisecond)
			results <- tache * 10
		}
	}
}

func main() {
	const nbWorkers = 3
	const nbTaches = 10

	ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
	defer cancel()

	jobs := make(chan int, nbTaches)
	results := make(chan int, nbTaches)
	var wg sync.WaitGroup

	// 1. Démarrage d'un nombre FIXE de Workers (3 goroutines max en parallèle)
	for w := 1; w <= nbWorkers; w++ {
		wg.Add(1)
		go worker(ctx, w, jobs, results, &wg)
	}

	// 2. Alimentation du canal de tâches par le producteur, puis fermeture !
	for j := 1; j <= nbTaches; j++ {
		jobs <- j
	}
```

```
	close(jobs) // Indique aux workers qu'aucune nouvelle tâche n'arrivera

	// 3. Goroutine de coordination : attend que tous les workers aient fini,
puis ferme 'results'
	go func() {
		wg.Wait()
		close(results) // Permet de débloquer la boucle 'for range results'
ci-dessous !
	}()

	// 4. Collecte des résultats jusqu'à fermeture de 'results'
	for res := range results {
		fmt.Println("Résultat traité :", res)
	}
}
```

Synthèse : Quand utiliser un Channel vs un Mutex en Go ?

Critère	**Utiliser un Channel (`chan`)**	**Utiliser un Mutex (`sync.Mutex`)**
Cas d'usage	Transférer la propriété d'une donnée, distribuer des tâches (*Worker Pool*, *Pipeline*), gérer des signaux/timeouts (`select`).	Protéger un état interne en mémoire (cache `map`, compteur, configuration) au sein d'une `struct`.
Portée	Coordination et orchestration entre plusieurs goroutines.	Section critique locale ultra-rapide (quelques nanosecondes en mémoire).